TEXORA · SESSIONAL NOTES · LAB EXPERIMENT SHEET

C Programming Using Simple Statements and Expressions

The anatomy of a C program, its data types and operators — and the discipline of predicting a program's output by hand before the machine is allowed to agree with you.

CSE-2106 · FUNDAMENTALS OF COMPUTING SESSIONAL · EXPERIMENT 4 · LEVEL 2 TERM 1

1. Aim

To write, compile and run simple C programs built from declarations, assignments, input/output statements and arithmetic expressions, and to verify each output by hand calculation.

2. Concepts you need

- **Program skeleton:** preprocessor directive `#include <stdio.h>` (gives printf/scanf), then `int main(void) { … return 0; }`. Every statement ends with a semicolon.
- **Basic types:** `int` (whole numbers), `float/double` (fractions), `char` (one character). A variable must be declared before use.
- **I/O:** `printf` with format specifiers %d (int), %f (float), %.2f (float, 2 decimals), %c (char); `scanf("%d", &x)` — the & (address of) is not optional.
- **Integer division trap:** with both operands int, / discards the fraction: 7/2 is 3, not 3.5. The remainder operator % gives what is left: 7%2 is 1. One float operand makes the whole operation floating-point: 7.0/2 is 3.5.
- **Precedence:** parentheses first; then * / % together, left to right; then + −. Assignment = is a statement, not a comparison.

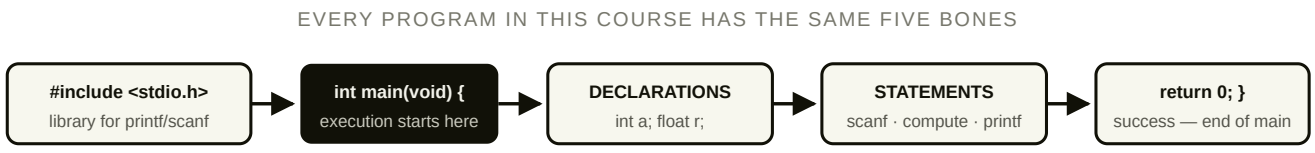


Figure 1 — Structure of a simple C program. Until functions arrive (Experiment 8), all work happens inside main, in statement order, top to bottom.

3. Lab procedure

1. Type each program exactly; save with a .c extension; compile. If the compiler reports an error, read the *first* error only, fix it, and recompile — later errors are often its echoes.
2. Before running, write your predicted output in the lab book. Run; compare. Any difference means your mental model — not "the computer" — needs the correction, and finding it is the experiment.
3. Evaluate the expression table in Section 4.3 by hand, then confirm two of them with a test program.

4. Worked examples

4.1 Area and circumference of a circle

```
#include <stdio.h>

int main(void)
{
    float r, area, circ;
    printf("Enter radius: ");
    scanf("%f", &r);
    area = 3.1416 * r * r;
    circ = 2 * 3.1416 * r;
    printf("Area = %.2f\n", area);
    printf("Circumference = %.2f\n", circ);
    return 0;
}
```

Output (input 7):

```
Enter radius: 7
Area = 153.94
Circumference = 43.98
```

Hand check: area = 3.1416 × 7² = 3.1416 × 49 = 153.94; circumference = 2 × 3.1416 × 7 = 43.98. ✓ Note %.2f rounds the display; the stored value keeps full precision.

4.2 Swapping two values

```
#include <stdio.h>

int main(void)
{
    int a = 15, b = 27, temp;
    temp = a;
    a = b;
    b = temp;
```

```
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

Output:

```
a = 27, b = 15
```

The third variable is essential: after a = b, the old a is gone forever. Assignment copies a value; it never "trades" anything.

4.3 Expressions — evaluate by hand first

EXPRESSION	VALUE	WHY
5 + 3 * 2	11	* before +
(5 + 3) * 2	16	parentheses first
17 % 5	2	17 = 3×5 + 2
7 / 2	3	both int → fraction discarded
7.0 / 2	3.5	one float operand → float result
9 * 37 / 5 + 32	98.6 (as float arithmetic)	the °C→°F formula: 333/5 = 66.6, +32 = 98.6 — but written with all-int literals it would give 66 + 32 = 98, the classic silent error

5. Practice tasks

1. Write a program that reads principal P, rate R (% per year) and time T (years) and prints the simple interest SI = P×R×T/100. Give the output for P = 5000, R = 6, T = 2.
2. Write a program that reads a length of time in minutes and prints it as hours and minutes. Give the output for 135 minutes, using / and %.
3. Predict, then explain: what does printf("%d", 10 / 4) print, and what single change makes it print 2.5?

Answer key

1.
```
#include <stdio.h>

int main(void)
{
    float p, r, t, si;
    printf("Enter principal, rate, time: ");
```

```
    scanf("%f %f %f", &p, &r, &t);
    si = p * r * t / 100;
    printf("Simple Interest = %.2f\n", si);
    return 0;
}
```

Output for 5000, 6, 2: SI = 5000 × 6 × 2 / 100 = **600.00**.

2.
```
#include <stdio.h>

int main(void)
{
    int mins, h, m;
    printf("Enter minutes: ");
    scanf("%d", &mins);
    h = mins / 60;
    m = mins % 60;
    printf("%d minutes = %d hours %d minutes\n", mins, h, m);
    return 0;
}
```

Output for 135: 135/60 = 2 (integer division), 135%60 = 15 → **"135 minutes = 2 hours 15 minutes"**.

3. It prints **2** — both operands are int, so 10/4 is integer division. Change one operand to a float literal and the specifier to %f: printf("%f", 10.0 / 4) prints 2.500000 (or %.1f to show 2.5).
